Computer Science
Complete Study Material

Internal Examinations — Comprehensive Guide

Topics: Data Structures | DBMS | Python | Networks | OS

Subject	**Computer Science**
Exam Type	**Internal Assessment**
Total Topics	**5 Major Units**
Important Questions	**50+ Questions Covered**
Difficulty Level	**Beginner to Advanced**

UNIT 1: DATA STRUCTURES

1.1 Introduction to Data Structures

A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. Data structures are essential for designing efficient algorithms and solving complex computational problems.

Types of Data Structures:

- **Linear Data Structures:** Arrays, Linked Lists, Stacks, Queues
- **Non-Linear Data Structures:** Trees, Graphs
- **Hash-Based Structures:** Hash Tables, Hash Maps
- **Heap Structures:** Min Heap, Max Heap, Priority Queue

1.2 Arrays

An array is a collection of elements stored at contiguous memory locations. It is the simplest and most widely used data structure.

Key Properties:

- Fixed size (static array) or dynamic size (dynamic array)
- Random access: O(1) time complexity
- Insertion/Deletion: O(n) in worst case
- Memory efficient — no extra pointers needed

Operation	Time Complexity	Space Complexity
Access	O(1)	O(1)
Search	O(n)	O(1)
Insertion	O(n)	O(1)
Deletion	O(n)	O(1)

1.3 Linked Lists

A linked list is a linear data structure where elements (nodes) are stored in non-contiguous memory locations. Each node contains data and a pointer/reference to the next node.

Types of Linked Lists:

- **Singly Linked List:** Each node points to next node only
- **Doubly Linked List:** Each node points to both next and previous
- **Circular Linked List:** Last node points back to first node

IMPORTANT: Linked List vs Array — Linked lists allow O(1) insertion at head, but O(n) random access. Arrays allow O(1) random access but O(n) insertion.

1.4 Stacks

A stack is a linear data structure that follows LIFO (Last In First Out) principle. The element inserted last is the first to be removed.

Operations: push(), pop(), peek(), isEmpty()

Applications: Function call stack, Undo operations, Expression evaluation, Browser history

Time Complexity: Push O(1), Pop O(1), Peek O(1)

1.5 Queues

A queue is a linear data structure that follows FIFO (First In First Out) principle. Elements are inserted at rear and removed from front.

Types: Simple Queue, Circular Queue, Priority Queue, Deque (Double Ended Queue)

Applications: CPU scheduling, Print spooling, BFS traversal, Handling requests

1.6 Trees

A tree is a non-linear hierarchical data structure consisting of nodes connected by edges. The topmost node is called root.

Tree Type	Key Property	Use Case
Binary Tree	Max 2 children per node	General hierarchy
BST	Left < Root < Right	Searching
AVL Tree	Balanced BST (height diff <=1)	Fast search/insert
Heap	Parent > Children (Max)	Priority Queue
B-Tree	Multi-way balanced tree	Database indexing

1.7 Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No

UNIT 2: DATABASE MANAGEMENT SYSTEMS (DBMS)

2.1 Introduction to DBMS

A Database Management System (DBMS) is software that manages databases. It provides an interface to perform various operations like creation, deletion, and modification of data. DBMS ensures data consistency, integrity, and security.

Advantages of DBMS:

- Eliminates data redundancy and inconsistency
- Provides data sharing among multiple users
- Ensures data integrity through constraints
- Provides data security and authorization
- Supports concurrent access with transaction management

2.2 ER Model (Entity-Relationship Model)

The ER Model is a conceptual data model used to describe the structure of a database. It uses entities, attributes, and relationships to represent real-world concepts.

- **Entity:** Real-world object (Student, Course, Teacher)
- **Attribute:** Property of an entity (Name, Age, ID)
- **Relationship:** Association between entities (Student ENROLLS Course)
- **Cardinality:** One-to-One, One-to-Many, Many-to-Many

2.3 Normalization

Normalization is the process of organizing data to reduce redundancy and improve data integrity. It involves dividing large tables into smaller ones and defining relationships.

Normal Form	Condition	Eliminates
1NF	Atomic values, no repeating groups	Duplicate rows
2NF	1NF + No partial dependency	Partial dependencies
3NF	2NF + No transitive dependency	Transitive dependencies
BCNF	3NF + Every determinant is candidate key	Anomalies in 3NF

2.4 SQL Commands

SQL (Structured Query Language) is used to communicate with databases.

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE	Define database structure
DML	SELECT, INSERT, UPDATE, DELETE	Manipulate data
DCL	GRANT, REVOKE	Control access permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

2.5 Transactions and ACID Properties

A transaction is a sequence of operations performed as a single logical unit of work. ACID properties ensure reliable transaction processing.

- **Atomicity:** All operations complete or none do (All or Nothing)
- **Consistency:** Database remains in consistent state before and after
- **Isolation:** Concurrent transactions don't interfere with each other
- **Durability:** Committed transactions persist even after system failure

IMPORTANT: ACID properties are the most frequently asked topic in DBMS exams. Remember: A-C-I-D stands for Atomicity, Consistency, Isolation, Durability.

2.6 Indexing

Indexing is a data structure technique used to quickly locate and access data. It improves the speed of data retrieval operations.

- **Primary Index:** Created on primary key (ordered file)
- **Secondary Index:** Created on non-primary key attributes
- **Clustered Index:** Data rows stored in order of index
- **B+ Tree Index:** Most commonly used in databases (MySQL, PostgreSQL)

UNIT 3: PYTHON PROGRAMMING

3.1 Python Basics

Python is a high-level, interpreted, general-purpose programming language. It emphasizes code readability with its notable use of significant indentation.

Key Features of Python:

- Simple and easy-to-learn syntax
- Dynamically typed — no need to declare variable types
- Interpreted language — no compilation needed
- Object-Oriented, Functional, and Procedural paradigms
- Extensive standard library and third-party packages

3.2 Data Types in Python

Data Type	Example	Mutable?	Description
int	42, -10, 0	No	Integer numbers
float	3.14, -2.5	No	Decimal numbers
str	"Hello", "CS"	No	Text/String
list	[1, 2, 3]	Yes	Ordered collection
tuple	(1, 2, 3)	No	Immutable ordered
dict	{"key": "val"}	Yes	Key-value pairs
set	{1, 2, 3}	Yes	Unique unordered
bool	True, False	No	Boolean values

3.3 Object-Oriented Programming (OOP)

OOP is a programming paradigm that organizes software design around data (objects) rather than functions and logic.

Four Pillars of OOP:

- **Encapsulation:** Bundling data and methods together, hiding internal details. Access modifiers: public, private (__), protected (_)
- **Inheritance:** Child class inherits properties from parent class. Types: Single, Multiple, Multilevel, Hierarchical, Hybrid
- **Polymorphism:** Same interface for different data types. Method overriding and operator overloading
- **Abstraction:** Hiding complex implementation, showing only necessary details. Achieved through abstract classes and interfaces

3.4 File Handling in Python

Python provides built-in functions to read, write, and manipulate files.

- open(file, mode) — modes: 'r' read, 'w' write, 'a' append, 'rb' binary read

- read(), readline(), readlines() — reading methods
- write(), writelines() — writing methods
- close() or use 'with' statement (context manager)

3.5 Exception Handling

Exception handling prevents program crashes by catching and handling runtime errors gracefully.

Syntax: try → except → else → finally

Common exceptions: ValueError, TypeError, KeyError, IndexError, FileNotFoundError

raise keyword: Used to manually throw exceptions

UNIT 4: COMPUTER NETWORKS

4.1 OSI Model

The OSI (Open Systems Interconnection) model is a conceptual framework with 7 layers that standardizes network communication functions.

Layer No.	Layer Name	Function	Protocols
7	Application	User interface, network services	HTTP, FTP, SMTP, DNS
6	Presentation	Data format, encryption	SSL, TLS, JPEG
5	Session	Session management	NetBIOS, RPC
4	Transport	End-to-end communication	TCP, UDP
3	Network	Logical addressing, routing	IP, ICMP, ARP
2	Data Link	Physical addressing (MAC)	Ethernet, WiFi
1	Physical	Bit transmission, hardware	Cables, Hubs

MEMORY TIP: Remember OSI layers from top to bottom: 'All People Seem To Need Data Processing' (Application, Presentation, Session, Transport, Network, Data Link, Physical)

4.2 TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (acknowledgment)	Unreliable
Speed	Slower	Faster
Flow Control	Yes	No
Error Checking	Yes	Minimal
Use Cases	HTTP, Email, FTP	Video streaming, DNS, Gaming
Header Size	20-60 bytes	8 bytes

4.3 IP Addressing

An IP address is a unique identifier assigned to each device connected to a network.

- **IPv4:** 32-bit address (4 octets) — Example: 192.168.1.1
- **IPv6:** 128-bit address (hexadecimal) — Solves IPv4 exhaustion
- **Subnet Mask:** Divides IP into network and host parts
- **CIDR Notation:** 192.168.1.0/24 means 24 bits for network

Class	Range	Default Subnet	Networks	Hosts/Networ
A	1.0.0.0 - 126.255.255.255	255.0.0.0	126	16,777,214
B	128.0.0.0 - 191.255.255.255	255.255.0.0	16,384	65,534
C	192.0.0.0 - 223.255.255.255	255.255.255.0	2,097,152	254

UNIT 5: OPERATING SYSTEMS

5.1 Introduction to Operating Systems

An Operating System (OS) is system software that manages computer hardware and software resources. It acts as an intermediary between users and computer hardware.

Functions: Process management, Memory management, File system, I/O management, Security

Types: Batch OS, Time-sharing OS, Real-time OS, Distributed OS, Network OS

5.2 Process Management

A process is a program in execution. The OS manages multiple processes concurrently.

Process States:

- **New:** Process being created
- **Ready:** Waiting to be assigned to processor
- **Running:** Instructions being executed
- **Waiting/Blocked:** Waiting for I/O or event
- **Terminated:** Process finished execution

5.3 CPU Scheduling Algorithms

Algorithm	Full Form	Preemptive?	Key Feature
FCFS	First Come First Serve	No	Simple, convoy effect
SJF	Shortest Job First	No	Minimum avg wait time
SRTF	Shortest Remaining Time First	Yes	Preemptive SJF
Round Robin	Round Robin	Yes	Time quantum, fairness
Priority	Priority Scheduling	Both	Starvation possible
MLQS	Multi-Level Queue	Both	Multiple queues

5.4 Memory Management

Memory management is the OS function responsible for allocating and managing RAM.

- **Paging:** Divides memory into fixed-size pages. No external fragmentation
- **Segmentation:** Divides memory into variable-size segments. Logical view
- **Virtual Memory:** Uses disk as extended RAM. Allows running larger programs
- **Page Replacement Algorithms:** FIFO, LRU, Optimal, LFU

5.5 Deadlocks

A deadlock is a situation where two or more processes are waiting for each other to release resources.

COFFMAN CONDITIONS (All 4 must hold for deadlock):

- **Mutual Exclusion:** Resources cannot be shared simultaneously
- **Hold and Wait:** Process holds resource while waiting for another

- **No Preemption:** Resources cannot be forcibly taken from processes
- **Circular Wait:** Circular chain of processes waiting for resources

IMPORTANT: Deadlock handling methods: Prevention (deny Coffman conditions), Avoidance (Banker's Algorithm), Detection and Recovery, Ignorance (Ostrich Algorithm)

IMPORTANT EXAM QUESTIONS

Unit 1: Data Structures — Important Questions

1. What is the difference between Array and Linked List? Explain with time complexity.
2. Explain Stack and Queue with real-life examples and operations.
3. What is Binary Search Tree (BST)? Explain insertion, deletion, and search operations.
4. Compare Bubble Sort, Merge Sort, and Quick Sort with time complexities.
5. What is a Hash Table? Explain collision resolution techniques.
6. Explain BFS and DFS graph traversal algorithms with examples.
7. What is the difference between Min Heap and Max Heap?
8. What are AVL Trees? Why are they used over simple BST?
9. Write an algorithm for binary search and explain its time complexity.
10. What is dynamic programming? Explain with an example (Fibonacci, Knapsack).

Unit 2: DBMS — Important Questions

1. What are ACID properties? Explain each with an example.
2. Explain all normal forms (1NF, 2NF, 3NF, BCNF) with examples.
3. What is the difference between DDL, DML, DCL, and TCL commands?
4. Explain ER diagram with cardinality and participation constraints.
5. What are joins in SQL? Explain INNER, LEFT, RIGHT, FULL OUTER JOIN.
6. What is indexing? Explain types of indexes and B+ Tree indexing.
7. What is concurrency control? Explain locking mechanisms.
8. What is a transaction? Explain commit, rollback, and savepoint.
9. Differentiate between clustered and non-clustered indexes.
10. What is SQL injection? How to prevent it?

Unit 3: Python — Important Questions

1. Explain the four pillars of OOP in Python with examples.
2. What is the difference between list, tuple, set, and dictionary?
3. What are decorators in Python? Explain with an example.
4. Explain exception handling with try, except, else, finally.
5. What is a lambda function? How is it different from regular function?
6. Explain list comprehension and generator expressions.
7. What is the difference between __init__ and __new__ methods?
8. What are Python modules and packages? How to import them?
9. Explain multithreading vs multiprocessing in Python.
10. What is the GIL (Global Interpreter Lock) in Python?

Unit 4: Computer Networks — Important Questions

1. Explain the OSI model with all 7 layers and their functions.
2. What is the difference between TCP and UDP? When to use each?
3. Explain IP addressing, subnetting, and CIDR notation.
4. What is DNS? Explain how it resolves domain names to IP addresses.
5. What is the three-way handshake in TCP connection establishment?
6. Explain HTTP vs HTTPS. What is SSL/TLS?
7. What is NAT (Network Address Translation) and why is it used?
8. Explain different network topologies: Star, Bus, Ring, Mesh.
9. What is the difference between hub, switch, and router?
10. Explain ARP and ICMP protocols with their uses.

Unit 5: Operating Systems — Important Questions

1. Explain all CPU scheduling algorithms with Gantt charts.
2. What are Coffman conditions for deadlock? How to prevent deadlock?
3. Explain paging and segmentation in memory management.
4. What is virtual memory? Explain page replacement algorithms.
5. What is the difference between process and thread?
6. Explain semaphores and mutex for process synchronization.
7. What is thrashing? How to prevent it?
8. Explain the Banker's Algorithm for deadlock avoidance.
9. What is context switching? What are its overheads?
10. What is a critical section problem? How is it solved?

QUICK REVISION — KEY FORMULAS & FACTS

Data Structures Quick Facts:

Concept	Key Point
Stack	LIFO — Last In First Out
Queue	FIFO — First In First Out
Binary Tree	Max nodes at level i = 2^i
Merge Sort	O(n log n) always — stable sort
Quick Sort	O(n log n) average — not stable
Hash Table	O(1) average for search/insert
Binary Search	O(log n) — needs sorted array
DFS	Uses Stack — depth first
BFS	Uses Queue — breadth first

OS Quick Facts:

Concept	Key Point
Process States	New → Ready → Running → Waiting → Terminated
FCFS	Non-preemptive, simple, convoy effect
Round Robin	Time quantum, most fair algorithm
Deadlock	4 Coffman conditions must hold simultaneously
Paging	Fixed size, no external fragmentation
Segmentation	Variable size, logical division
Thrashing	Too many page faults, CPU busy with paging
Semaphore	wait(P) and signal(V) operations

Networks Quick Facts:

Concept	Key Point
OSI Layer 7	Application — HTTP, FTP, SMTP, DNS
OSI Layer 4	Transport — TCP, UDP
OSI Layer 3	Network — IP, ICMP, ARP
TCP	Reliable, connection-oriented, 3-way handshake
UDP	Fast, connectionless, no guarantee
IPv4	32-bit address, 4.3 billion addresses
IPv6	128-bit, solves IPv4 exhaustion
DNS Port	Port 53
HTTP Port	Port 80, HTTPS Port 443